TP guidé de préparation au stage

Pipeline Data Science complet sur `Housing.csv`

De l'exploration au modèle industrialisable

Préparation stage : prédiction de saturation PBO FTTH

Version de travail

Table des matières

1 Avis sur la cohérence du manuel **3**

2 Sujet de TP proposé **3**
- 2.1 Titre . . . 3
- 2.2 Pourquoi ce TP est cohérent avec le stage PBO ? . . . 3
- 2.3 Description du dataset . . . 3

3 Objectifs pédagogiques **4**

4 Normes professionnelles à respecter **4**
- 4.1 Quand utiliser un notebook `.ipynb` ? . . . 4
- 4.2 Quand utiliser des fichiers `.py` ? . . . 4
- 4.3 Quand créer une classe ? . . . 5
- 4.4 Structure de projet attendue . . . 5

5 Règles mathématiques et probabilistes à expliciter dans le TP **6**
- 5.1 Standardisation centrée-réduite . . . 6
- 5.2 Corrélation linéaire de Pearson . . . 6
- 5.3 Séparation train/test et estimation de généralisation . . . 7
- 5.4 Validation croisée K-fold . . . 7
- 5.5 Moindres carrés ordinaires pour la régression linéaire . . . 7
- 5.6 Métriques de régression : MAE, RMSE et R^2 . . . 8
- 5.7 Matrice de confusion et métriques de classification . . . 8
- 5.8 Déséquilibre de classes et stratification . . . 8
- 5.9 Distance de Haversine pour le voisinage géographique . . . 9

6 Déroulement du TP **9**
- 6.1 Étape 0 – Installation et imports . . . 9
- 6.2 Étape 1 – Chargement et audit initial . . . 9
- 6.3 Étape 2 – Analyse exploratoire univariée . . . 10
- 6.4 Étape 3 – Analyse bivariée et intuition métier . . . 11
- 6.5 Étape 4 – Préparation des données sans fuite . . . 11
- 6.6 Étape 5 – Baseline de régression . . . 12
- 6.7 Étape 6 – Modèles plus avancés . . . 13
- 6.8 Étape 7 – Transformation en problème de classification . . . 14
- 6.9 Étape 8 – Classification et métriques . . . 14
- 6.10 Étape 9 – Feature engineering . . . 15
- 6.11 Étape 10 – Sauvegarde du modèle . . . 16

7 Travail demandé au stagiaire **16**
7.1 Partie A – Notebook d'exploration 16
7.2 Partie B – Notebook de modélisation 17
7.3 Partie C – Code Python propre 17
7.4 Partie D – Synthèse finale 17

8 Grille d'évaluation **17**

9 Extensions pour se rapprocher du stage PBO **17**

10 Checklist finale **18**

11 Conclusion générale **18**

1 Avis sur la cohérence du manuel

Objectif

Le manuel est cohérent pour une préparation au stage : il couvre les briques nécessaires à un projet de prédiction de saturation PBO FTTH, à savoir **NumPy**, **Pandas**, **Matplotlib/Seaborn**, **Scikit-learn**, **XGBoost**, les notions mathématiques utiles, les métriques, les pipelines, et une partie géospatiale.

Le sujet officiel du stage demande de prédire la saturation des PBO à partir de données historiques et de la proximité géographique des équipements. Les travaux attendus sont : préparation des données, construction d'un historique par PBO, calcul des voisins géographiques, feature engineering, entraînement d'un modèle XGBoost, prédiction et livrables de code/modèle/rapport. Le manuel va donc dans la bonne direction.

Point fort. Le manuel ne se limite pas aux bibliothèques : il insiste aussi sur la méthodologie, les métriques, les fuites de données, les pipelines et l'interprétation.

Point à améliorer pour ta préparation. Le manuel est très riche, mais pour progresser il faut maintenant passer d'une lecture chapitre par chapitre à un **TP intégrateur unique**, où tu révises toutes les bases sur un dataset fourni, avec une organisation proche d'un vrai projet de stage.

2 Sujet de TP proposé

2.1 Titre

TP – Pipeline complet de prédiction supervisée sur données tabulaires Application : prédiction du prix immobilier et simulation d'un risque binaire

2.2 Pourquoi ce TP est cohérent avec le stage PBO ?

Le fichier `Housing.csv` n'est pas un dataset FTTH, mais il permet de travailler exactement les compétences techniques nécessaires au stage :

- données tabulaires mixtes : variables numériques et catégorielles ;
- nettoyage, typage, encodage, analyse exploratoire ;
- construction de variables explicatives ;
- séparation train/test et validation croisée ;
- pipelines Scikit-learn ;
- comparaison de modèles ;
- passage d'un notebook exploratoire à des fichiers `.py` réutilisables ;
- production de conclusions métier.

2.3 Description du dataset

Le dataset contient 545 lignes et 13 colonnes. La cible principale est `price`. Les variables disponibles sont :

Colonne	Type / rôle
`price`	cible de régression
`area`	surface
`bedrooms`, `bathrooms`, `stories`	caractéristiques numériques
`mainroad`, `guestroom`, `basement`	variables oui/non
`hotwaterheating`, `airconditioning`	variables oui/non
`parking`	nombre de places
`prefarea`	zone préférée oui/non
`furnishingstatus`	variable catégorielle à plusieurs modalités

3 Objectifs pédagogiques

À la fin du TP, tu dois savoir :

1. charger et auditer un dataset avec Pandas ;
2. distinguer variables numériques, catégorielles, cible, features ;
3. produire une analyse exploratoire structurée ;
4. créer un pipeline de preprocessing sans fuite de données ;
5. entraîner une baseline simple puis des modèles plus puissants ;
6. évaluer un modèle de régression avec MAE, RMSE et R^2 ;
7. transformer un problème de régression en classification binaire ;
8. évaluer une classification avec précision, rappel, F1, ROC-AUC ;
9. sauvegarder un modèle et écrire un script de prédiction ;
10. organiser le projet comme un mini-projet professionnel.

4 Normes professionnelles à respecter

4.1 Quand utiliser un notebook `.ipynb` ?

Norme professionnelle

Un notebook est adapté pour l'exploration, les visualisations, les tests rapides et la narration pédagogique. Il ne doit pas devenir le seul endroit où vit le code final.

Utilise un notebook pour :

— inspecter les données ;
— tester des hypothèses ;
— afficher des graphiques ;
— comparer rapidement des modèles ;
— rédiger les observations intermédiaires.

Nom recommandé : `notebooks/01_eda_housing.ipynb` puis `notebooks/02_modeling_housing.ipynb`.

4.2 Quand utiliser des fichiers `.py` ?

Norme professionnelle

Dès qu'un morceau de code doit être relancé, testé, réutilisé ou livré, il doit sortir du notebook et aller dans un fichier `.py`.

Utilise des fichiers `.py` pour :

- charger les données ;
- préparer les features ;
- définir le pipeline ;
- entraîner le modèle ;
- sauvegarder le modèle ;
- lancer une prédiction sur de nouvelles données.

4.3 Quand créer une classe ?

Norme professionnelle

En data science, on ne met pas tout en classe. Une classe est utile quand tu représentes un composant stable avec un état ou une API claire. Pour le reste, des fonctions simples sont préférables.

Situation	Recommandation
Exploration ponctuelle	Notebook, pas de classe
Nettoyage simple	Fonctions
Calcul de métriques	Fonctions
Pipeline Scikit-learn	`Pipeline` et `ColumnTransformer`
Transformation personnalisée réutilisable	Classe compatible Scikit-learn
Application de prédiction complète	Classe possible : `HousingPricePredictor`

4.4 Structure de projet attendue

```
housing_tp/
|-- data/
|   `-- Housing.csv
|-- notebooks/
|   |-- 01_eda_housing.ipynb
|   `-- 02_modeling_housing.ipynb
|-- src/
|   |-- data.py
|   |-- features.py
|   |-- model.py
|   |-- evaluate.py
|   `-- predict.py
|-- models/
|   `-- housing_model.joblib
|-- reports/
|   |-- figures/
|   `-- synthese_tp.md
|-- requirements.txt
`-- README.md
```

5 Règles mathématiques et probabilistes à expliciter dans le TP

Attention

Dans ce TP, dès qu'une règle mathématique, statistique ou probabiliste est utilisée, il faut écrire trois éléments :

1. le nom de la règle ou du théorème ;
2. la formule ou l'idée mathématique ;
3. l'objectif pratique : pourquoi on l'utilise dans le projet.

Cela évite d'appliquer des outils comme des boîtes noires et prépare mieux au rapport de stage.

5.1 Standardisation centrée-réduite

Théorème / règle mathématique

Pour une variable numérique X, la standardisation transforme chaque valeur x_i en score réduit :

$$z_i = \frac{x_i - \bar{x}}{s}$$

où $\bar{x}$ est la moyenne empirique et s l'écart-type empirique.

Objectif de la règle

Objectif : mettre les variables numériques sur une échelle comparable. C'est important pour les modèles sensibles aux distances ou aux coefficients, par exemple kNN, régression linéaire régularisée, régression logistique et SVM. Dans le TP, on l'utilise dans le `ColumnTransformer` uniquement sur le jeu d'entraînement pour éviter la fuite de données.

5.2 Corrélation linéaire de Pearson

Théorème / règle mathématique

La corrélation de Pearson entre deux variables X et Y est :

$$\rho_{X,Y} = \frac{\mathrm{Cov}(X,Y)}{\sigma_X \sigma_Y}$$

avec une valeur comprise entre -1 et 1.

Objectif de la règle

Objectif : mesurer l'intensité d'une relation linéaire entre deux variables. Dans le TP Housing, elle aide à repérer les variables liées au prix. Dans le stage PBO, elle pourra aider à vérifier si la saturation est liée au taux d'occupation, à l'ancienneté, au nombre de clients ou aux variables de voisinage. Attention : corrélation ne signifie pas causalité.

5.3 Séparation train/test et estimation de généralisation

Théorème / règle mathématique

On sépare les données en deux ensembles :

$$D = D_{train} \cup D_{test}, \qquad D_{train} \cap D_{test} = \varnothing$$

Le modèle apprend sur D_{train} et il est évalué une seule fois sur D_{test}.

Objectif de la règle

Objectif : estimer la capacité du modèle à généraliser sur de nouvelles données. Le test set simule les données futures. Dans le stage PBO, c'est essentiel car le modèle devra prédire des saturations non encore observées.

5.4 Validation croisée K-fold

Théorème / règle mathématique

En validation croisée K-fold, les données sont découpées en K blocs. À chaque itération, on entraîne sur $K-1$ blocs et on valide sur le bloc restant :

$$\widehat{R}_{CV} = \frac{1}{K}\sum_{k=1}^{K} R_k$$

où R_k est l'erreur mesurée sur le pli k.

Objectif de la règle

Objectif : obtenir une évaluation plus robuste qu'un seul split train/test, surtout lorsque le dataset est petit. Dans le TP Housing, cela permet de comparer les modèles. Dans le stage PBO, cela aidera à choisir entre baseline, random forest et XGBoost.

5.5 Moindres carrés ordinaires pour la régression linéaire

Théorème / règle mathématique

La régression linéaire cherche à minimiser l'erreur quadratique :

$$L(w,b) = \frac{1}{n}\sum_{i=1}^{n}(y_i - w^T x_i - b)^2$$

Sous certaines conditions, la solution analytique s'écrit :

$$\hat{w} = (X^T X)^{-1} X^T y$$

Objectif de la règle

Objectif : construire une baseline simple, interprétable et rapide. Dans le TP, elle sert de référence avant les modèles plus complexes. Dans le stage PBO, l'équivalent classification sera souvent la régression logistique comme première baseline.

5.6 Métriques de régression : MAE, RMSE et R^2

Théorème / règle mathématique

Pour des prédictions $\hat{y}_i$ et des vraies valeurs y_i :

$$MAE = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

$$RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$$

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}$$

Objectif de la règle

Objectif : interpréter l'erreur du modèle. La MAE donne une erreur moyenne facile à comprendre dans l'unité du prix. La RMSE pénalise davantage les grosses erreurs. Le R^2 indique la part de variance expliquée par le modèle.

5.7 Matrice de confusion et métriques de classification

Théorème / règle mathématique

Pour une classification binaire :

$$\text{Précision} = \frac{TP}{TP + FP}, \qquad \text{Rappel} = \frac{TP}{TP + FN}$$

$$F1 = 2 \times \frac{\text{Précision} \times \text{Rappel}}{\text{Précision} + \text{Rappel}}$$

où TP sont les vrais positifs, FP les faux positifs et FN les faux négatifs.

Objectif de la règle

Objectif : choisir une métrique adaptée au risque métier. Pour la saturation PBO, le rappel de la classe saturée est important si l'on veut éviter de rater un PBO réellement à risque. La précision est importante si les fausses alertes coûtent cher aux équipes opérationnelles.

5.8 Déséquilibre de classes et stratification

Théorème / règle mathématique

Lorsque la classe positive est rare, l'accuracy peut être trompeuse. Par exemple, si 95% des PBO ne sont pas saturés, un modèle qui prédit toujours "non saturé" obtient 95% d'accuracy mais ne détecte aucun risque.

Objectif de la règle

Objectif : forcer une évaluation honnête. On utilise `stratify=y` lors du split pour conserver les proportions de classes, et on regarde le rappel, la précision, le F1-score, la ROC-AUC ou la PR-AUC plutôt que l'accuracy seule.

5.9 Distance de Haversine pour le voisinage géographique

Théorème / règle mathématique

Pour deux points de coordonnées (φ_1, λ_1) et (φ_2, λ_2) en radians, la distance de Haversine est :

$$d = 2R \arcsin\left(\sqrt{\sin^2\left(\frac{\Delta\varphi}{2}\right) + \cos(\varphi_1)\cos(\varphi_2)\sin^2\left(\frac{\Delta\lambda}{2}\right)}\right)$$

où $R \approx 6371$ km.

Objectif de la règle

Objectif : calculer des distances réalistes entre équipements géographiques. Dans le stage PBO, cette règle sert à construire les voisins les plus proches et des variables comme la saturation moyenne des voisins.

6 Déroulement du TP

6.1 Étape 0 – Installation et imports

Objectif

Préparer un environnement reproductible et importer les bibliothèques nécessaires.

```
python -m venv .venv
source .venv/bin/activate       # Linux / Mac
# .venv\Scripts\activate        # Windows
pip install pandas numpy matplotlib seaborn scikit-learn xgboost joblib
    jupyter
```

Dans `requirements.txt`, indiquer au minimum :

```
pandas
numpy
matplotlib
seaborn
scikit-learn
xgboost
joblib
jupyter
```

Conclusion à tirer

Un projet sérieux doit pouvoir être relancé sur une autre machine. Le fichier `requirements.txt` est donc un livrable technique, pas un détail.

6.2 Étape 1 – Chargement et audit initial

Objectif

Comprendre la structure du dataset avant toute modélisation.

```
import pandas as pd
```

```
DATA_PATH = "data/Housing.csv"
df = pd.read_csv(DATA_PATH)

print(df.shape)
print(df.head())
print(df.info())
print(df.describe(include="all"))
print(df.isna().sum())
```

Questions à traiter :

1. Combien y a-t-il de lignes et de colonnes ?
2. Quelle est la cible ?
3. Quelles colonnes sont numériques ?
4. Quelles colonnes sont catégorielles ?
5. Y a-t-il des valeurs manquantes ?
6. Y a-t-il des valeurs aberrantes visibles ?

Livrable attendu

Un tableau de synthèse contenant pour chaque colonne : type, nombre de valeurs manquantes, nombre de modalités si catégorielle, rôle dans le modèle.

Conclusion à tirer

Avant de prédire, on doit savoir ce que représente chaque variable. Dans le stage PBO, cette étape correspondra à la compréhension du schéma des données FTTH : PBO, clients, dates, coordonnées, capacité, occupation, historique.

6.3 Étape 2 – Analyse exploratoire univariée

Objectif

Étudier chaque variable séparément pour repérer distributions, déséquilibres et anomalies.

```
import matplotlib.pyplot as plt
import seaborn as sns

num_cols = ["price", "area", "bedrooms", "bathrooms", "stories", "parking"
    ]
cat_cols = ["mainroad", "guestroom", "basement", "hotwaterheating",
            "airconditioning", "prefarea", "furnishingstatus"]

for col in num_cols:
    fig, ax = plt.subplots(figsize=(6, 4))
    sns.histplot(df[col], kde=True, ax=ax)
    ax.set_title(f"Distribution de {col}")
    plt.tight_layout()
    plt.show()

for col in cat_cols:
    print(df[col].value_counts(normalize=True))
```

Questions à traiter :

1. La variable `price` est-elle symétrique ou asymétrique ?

2. Y a-t-il beaucoup de maisons avec chauffage eau chaude ?
3. Certaines modalités sont-elles rares ?
4. La surface `area` contient-elle des valeurs extrêmes ?

Conclusion à tirer

Une variable très déséquilibrée ou très asymétrique peut influencer le modèle. Pour le stage PBO, la saturation sera probablement une classe rare : il faudra donc être attentif au déséquilibre de classes.

6.4 Étape 3 – Analyse bivariée et intuition métier

Objectif

Comprendre quelles variables semblent liées au prix. Voir aussi la règle sur la corrélation de Pearson dans la section précédente.

```
# Corrélations numériques
corr = df[num_cols].corr()
print(corr["price"].sort_values(ascending=False))

fig, ax = plt.subplots(figsize=(7, 5))
sns.heatmap(corr, annot=True, cmap="coolwarm", center=0, ax=ax)
ax.set_title("Matrice de corrélation")
plt.tight_layout()
plt.show()

# Prix selon variables catégorielles
for col in cat_cols:
    fig, ax = plt.subplots(figsize=(6, 4))
    sns.boxplot(data=df, x=col, y="price", ax=ax)
    ax.set_title(f"Prix selon {col}")
    plt.xticks(rotation=30)
    plt.tight_layout()
    plt.show()
```

Questions à traiter :

1. Quelle variable numérique est la plus corrélée au prix ?
2. La climatisation semble-t-elle augmenter le prix ?
3. La zone préférée semble-t-elle importante ?
4. Quels graphiques seraient utiles dans un rapport de stage ?

Conclusion à tirer

La corrélation ne prouve pas la causalité, mais elle aide à formuler des hypothèses. Dans le stage PBO, on cherchera de la même façon si la saturation est liée au nombre de clients, à l'ancienneté, à la densité de voisins ou au taux d'occupation.

6.5 Étape 4 – Préparation des données sans fuite

Objectif

Créer un pipeline propre qui apprend les transformations uniquement sur le train. Voir aussi les règles sur la standardisation et la séparation train/test.

Attention

Ne jamais faire l'encodage, la standardisation ou l'imputation sur tout le dataset avant le split train/test. Cela crée une fuite de données.

```
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.pipeline import Pipeline

X = df.drop(columns=["price"])
y = df["price"]

numeric_features = ["area", "bedrooms", "bathrooms", "stories", "parking"]
categorical_features = ["mainroad", "guestroom", "basement", "
    hotwaterheating",
                        "airconditioning", "prefarea", "furnishingstatus"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

preprocessor = ColumnTransformer(
    transformers=[
        ("num", StandardScaler(), numeric_features),
        ("cat", OneHotEncoder(handle_unknown="ignore"),
            categorical_features),
    ]
)
```

Conclusion à tirer

Le pipeline est une norme professionnelle : il évite les erreurs silencieuses et rend le modèle réutilisable en production.

6.6 Étape 5 – Baseline de régression

Objectif

Construire un premier modèle simple pour avoir un point de comparaison. Voir aussi la règle des moindres carrés et les métriques MAE/RMSE/R^2.

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error,
    r2_score
import numpy as np

baseline = Pipeline(steps=[
    ("preprocess", preprocessor),
    ("model", LinearRegression())
])

baseline.fit(X_train, y_train)
y_pred = baseline.predict(X_test)

mae = mean_absolute_error(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
```

```
r2 = r2_score(y_test, y_pred)

print(f"MAE  : {mae:.2f}")
print(f"RMSE : {rmse:.2f}")
print(f"R2   : {r2:.3f}")
```

Questions à traiter :

1. Que signifie la MAE dans l'unité du problème ?
2. Est-ce que le R^2 est satisfaisant ?
3. Le modèle sous-apprend-il ou sur-apprend-il ?

Conclusion à tirer

Une baseline est indispensable. Dans le stage PBO, avant XGBoost, il faudra aussi entraîner un modèle simple : régression logistique, arbre de décision ou random forest simple.

6.7 Étape 6 – Modèles plus avancés

Objectif

Comparer plusieurs modèles pour comprendre le compromis biais-variance. Voir aussi la validation croisée K-fold.

```
from sklearn.ensemble import RandomForestRegressor,
    GradientBoostingRegressor
from sklearn.model_selection import cross_validate

models = {
    "linear_regression": LinearRegression(),
    "random_forest": RandomForestRegressor(
        n_estimators=300, random_state=42, n_jobs=-1
    ),
    "gradient_boosting": GradientBoostingRegressor(random_state=42)
}

for name, model in models.items():
    pipe = Pipeline(steps=[
        ("preprocess", preprocessor),
        ("model", model)
    ])
    scores = cross_validate(
        pipe, X_train, y_train,
        cv=5,
        scoring={
            "mae": "neg_mean_absolute_error",
            "r2": "r2"
        }
    )
    print(name)
    print("MAE CV:", -scores["test_mae"].mean())
    print("R2 CV :", scores["test_r2"].mean())
```

Option XGBoost :

```
from xgboost import XGBRegressor

xgb_model = XGBRegressor(
    n_estimators=500,
```

```
    max_depth=3,
    learning_rate=0.05,
    subsample=0.8,
    colsample_bytree=0.8,
    objective="reg:squarederror",
    random_state=42,
    n_jobs=-1
)

xgb_pipe = Pipeline(steps=[
    ("preprocess", preprocessor),
    ("model", xgb_model)
])
```

Conclusion à tirer

Un modèle complexe doit être justifié par une amélioration mesurable. En entreprise, on ne choisit pas XGBoost parce qu'il est populaire, mais parce qu'il améliore une métrique utile.

6.8 Étape 7 – Transformation en problème de classification

Objectif

Préparer la logique du stage PBO : prédire un état binaire, par exemple saturé / non saturé.

Dans le dataset Housing, on simule une cible binaire : `expensive`. Une maison est considérée comme chère si son prix est dans les 25% les plus élevés.

```
threshold = df["price"].quantile(0.75)
df["expensive"] = (df["price"] >= threshold).astype(int)

X_clf = df.drop(columns=["price", "expensive"])
y_clf = df["expensive"]

X_train, X_test, y_train, y_test = train_test_split(
    X_clf, y_clf,
    test_size=0.2,
    random_state=42,
    stratify=y_clf
)
```

Attention

On utilise `stratify=y_clf` pour garder la même proportion de classes dans train et test. C'est très important pour les problèmes déséquilibrés, comme la saturation PBO.

6.9 Étape 8 – Classification et métriques

Objectif

Évaluer un modèle binaire avec les bonnes métriques. Voir aussi la matrice de confusion, le rappel, la précision et le F1-score.

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, confusion_matrix,
    roc_auc_score
```

```
clf = Pipeline(steps=[
    ("preprocess", preprocessor),
    ("model", LogisticRegression(max_iter=2000, class_weight="balanced"))
])

clf.fit(X_train, y_train)
y_pred = clf.predict(X_test)
y_proba = clf.predict_proba(X_test)[:, 1]

print(confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred))
print("ROC-AUC:", roc_auc_score(y_test, y_proba))
```

Questions à traiter :

1. Si la classe positive est rare, pourquoi l'accuracy peut-elle être trompeuse ?
2. Dans le stage PBO, vaut-il mieux rater un PBO saturé ou déclencher une fausse alerte ?
3. Quelle métrique privilégier : précision, rappel, F1, PR-AUC ?

Conclusion à tirer

Pour un sujet de saturation, le rappel de la classe saturée est souvent crucial : il mesure la capacité à détecter les vrais risques. La précision reste importante pour éviter trop de fausses alertes.

6.10 Étape 9 – Feature engineering

Objectif

Créer des variables utiles à partir des colonnes brutes.

Exemples sur `Housing.csv` :

```
def add_features(df):
    df = df.copy()
    df["rooms_total"] = df["bedrooms"] + df["bathrooms"]
    df["area_per_room"] = df["area"] / df["rooms_total"].replace(0, 1)
    df["has_luxury_option"] = (
        (df["airconditioning"] == "yes") |
        (df["prefarea"] == "yes") |
        (df["guestroom"] == "yes")
    ).astype(int)
    return df
```

Norme professionnelle

Une feature doit être disponible au moment de la prédiction. Dans le stage PBO, ne jamais utiliser une information future pour prédire une saturation passée ou présente.

Conclusion à tirer

Le feature engineering est le coeur du stage. Pour les PBO, les features importantes seront par exemple : nombre de clients actifs, historique de raccordements, taux d'occupation, densité de voisins, moyenne de saturation des voisins, distance au PBO le plus proche.

6.11 Étape 10 – Sauvegarde du modèle

Objectif

Produire un modèle réutilisable sans relancer tout le notebook.

```
import joblib

final_model = clf
final_model.fit(X_clf, y_clf)
joblib.dump(final_model, "models/housing_classifier.joblib")
```

Script `src/predict.py` :

```
import argparse
import joblib
import pandas as pd


def main(input_path, model_path, output_path):
    model = joblib.load(model_path)
    data = pd.read_csv(input_path)
    proba = model.predict_proba(data)[:, 1]
    pred = model.predict(data)
    data["proba_expensive"] = proba
    data["prediction_expensive"] = pred
    data.to_csv(output_path, index=False)


if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--input", required=True)
    parser.add_argument("--model", required=True)
    parser.add_argument("--output", required=True)
    args = parser.parse_args()
    main(args.input, args.model, args.output)
```

Commande attendue :

```
python src/predict.py --input data/Housing.csv --model models/
   housing_classifier.joblib --output reports/predictions.csv
```

Conclusion à tirer

Le passage notebook → script est une étape clé de professionnalisation. Dans le stage, le livrable ne sera pas seulement un notebook, mais aussi un script de prédiction exploitable.

7 Travail demandé au stagiaire

7.1 Partie A – Notebook d'exploration

Dans `01_eda_housing.ipynb` :

1. charger les données ;
2. afficher types, dimensions, valeurs manquantes ;
3. produire au moins 6 graphiques ;
4. commenter chaque graphique ;
5. identifier les variables qui semblent les plus utiles.

7.2 Partie B – Notebook de modélisation

Dans `02_modeling_housing.ipynb` :

1. créer un pipeline complet ;
2. entraîner une baseline ;
3. entraîner au moins deux modèles avancés ;
4. comparer les scores en validation croisée ;
5. évaluer le meilleur modèle sur le test set ;
6. interpréter les résultats.

7.3 Partie C – Code Python propre

Dans `src/` :

— `data.py` : fonction `load_data(path)` ;
— `features.py` : fonction `add_features(df)` ;
— `model.py` : fonction `build_pipeline(model_type)` ;
— `evaluate.py` : fonctions de métriques ;
— `predict.py` : script de prédiction.

7.4 Partie D – Synthèse finale

Dans `reports/synthese_tp.md`, rédiger une page contenant :

1. objectif du TP ;
2. description des données ;
3. principales observations EDA ;
4. modèles testés ;
5. meilleurs résultats ;
6. limites ;
7. parallèle avec le stage PBO.

8 Grille d'évaluation

Critère	Points
Audit clair du dataset	2
Analyse exploratoire commentée	3
Pipeline sans fuite de données	4
Comparaison rigoureuse de modèles	3
Utilisation correcte des métriques	3
Feature engineering justifié	2
Code `.py` propre et réutilisable	3
Synthèse claire et lien avec le stage	3
Organisation du projet	2
Total	25

9 Extensions pour se rapprocher du stage PBO

Une fois le TP Housing terminé, tu peux créer un second mini-dataset simulé PBO :

```python
import numpy as np
import pandas as pd

rng = np.random.default_rng(42)
n = 1000
pbo = pd.DataFrame({
    "pbo_id": [f"PBO_{i:04d}" for i in range(n)],
    "latitude": rng.normal(48.85, 0.05, n),
    "longitude": rng.normal(2.35, 0.05, n),
    "capacity": rng.choice([6, 8, 12], size=n),
    "nb_clients": rng.poisson(5, size=n),
    "age_days": rng.integers(30, 2000, size=n),
})
pbo["occupation_rate"] = pbo["nb_clients"] / pbo["capacity"]
pbo["sature"] = (pbo["occupation_rate"] >= 0.9).astype(int)
```

Travail complémentaire :

1. calculer les voisins géographiques ;
2. ajouter `mean_occupation_neighbors` ;
3. entraîner une classification binaire ;
4. comparer les résultats avec et sans variables de voisinage.

Conclusion à tirer

Cette extension fait le pont direct avec le stage : elle introduit la cible `sature`, les coordonnées géographiques, le voisinage et la logique de prédiction de risque.

10 Checklist finale

Avant de considérer le TP comme terminé, vérifier :

- le projet se lance après installation de `requirements.txt` ;
- les chemins de fichiers sont relatifs ;
- aucun preprocessing n'est appris sur le test set ;
- les notebooks sont lisibles et commentés ;
- les scripts `.py` fonctionnent hors notebook ;
- les métriques sont interprétées, pas seulement affichées ;
- les conclusions sont reliées à un besoin métier ;
- le modèle final est sauvegardé ;
- le script de prédiction produit un CSV de sortie.

11 Conclusion générale

Ce TP sert à transformer la théorie du manuel en compétence pratique. Il ne remplace pas le sujet PBO, mais il prépare les gestes techniques indispensables : nettoyer, explorer, construire des features, entraîner, évaluer, interpréter et livrer un modèle. Pour le stage, il faudra ensuite remplacer la cible `price/expensive` par la cible `sature`, et remplacer les variables immobilières par les variables FTTH : historique clients, capacité, occupation, ancienneté, coordonnées et voisinage géographique.